Bean Beacon — Final Documentation

- Bean Beacon — Final Documentation
  - Part I — Overview
  - 1. Project Presentation
  - 2. System Architecture
  - 3. Repository Structure
  - 4. DevOps Architecture Diagrams
  - Part II — Getting Started
  - 5. Prerequisites
  - 6. Running with Docker Compose
  - 7. Manual Setup (without Docker)
  - 8. Tests and Validation
  - Part III — Application Logic
  - 9. Frontend Architecture
  - 10. Backend Architecture
  - 11. Recommendation Logic
  - Part IV — API Reference
  - 12. Authentication
  - 13. Cafés
  - 14. Favorites
  - 15. Ratings & Reviews
  - 16. Admin
  - 17. Complete Route Table
  - 18. Error Handling
  - Part V — CI/CD and Deployment
  - 19. Git and GitLab Workflow
  - 20. GitLab CI/CD Pipeline
  - 21. Production Deployment
  - 22. Environment Variables
  - Part VI — Observability
  - 23. Monitoring Stack
  - Part VII — Reference
  - 24. Feature Improvement Backlog
  - 25. Known Risks and Gaps
  - 26. Troubleshooting
  - 27. Final Deliverables

Bean Beacon — Final Documentation

Version 3.0 — consolidated reference

This document supersedes and merges `README`-level guides, `ARCHITECTURE.md`, `API.md`, `CURRENT_STATUS.md`, `DEPLOYMENT.md`, `DEVELOPMENT.md`, `DEVOPS.md`, `SETUP.md`, `RUN.md`, both versions of the *Cahier des charges*, and the *Feature Improvement Ideas* note into a single source of truth. Where earlier documents disagreed, this version follows the application's source code and Compose files, since those are authoritative.

Part I — Overview

1. Project Presentation

Bean Beacon is a web application for discovering coffee shops. It combines:

- a public café directory with search, category, and attribute filtering;
- an interactive map (Leaflet + OpenStreetMap tiles) with Google Maps directions links;
- café profiles and community reviews;
- user-submitted café listings with admin moderation;
- a preference-based recommendation feature (deterministic token matching, not ML);
- JWT authentication with role-based access (user/admin);

- Docker-based local and production deployment;
- Prometheus, Grafana, and Uptime Kuma observability.

The existing React/Express/MongoDB architecture is the one actually implemented and maintained. Any earlier references to a Flask-based example stack do not apply to this project.

Layer	Technology
Frontend	React 18 + Vite + TypeScript, React Router, TanStack Query
Backend	Node.js + Express + TypeScript
Database	MongoDB + Mongoose
Containers	Docker Compose (dev and prod)
CI/CD	GitLab Pipeline
Monitoring	Prometheus, Grafana, Uptime Kuma
Map data	OpenStreetMap, Overpass API, Nominatim
Security	JWT, bcrypt, Gitleaks secret scanning

2. System Architecture

```
Browser
  | React 18 + Vite + React Router + TanStack Query
  | fetch() with optional Bearer JWT
  v
Frontend
  | development: Vite :8081
  | production: Nginx :80 (/api/* reverse-proxied to backend)
  v
Express API :3000
  |-- JWT middleware / optional-auth / admin guard
  |-- controllers (auth, cafes, favorites, ratings, admin)
  |-- Mongoose models (User, Cafe, Favorite, Rating)
  v
MongoDB :27017
  |-- 2dsphere index on Cafe.location
  |-- compound unique indexes (Favorite, Rating)

External services:
  Overpass API  — radius café discovery + OSM import
  Nominatim     — address geocoding for submitted cafés
  Google Maps   — directions links only (not the map renderer)

Observability:
  Backend /metrics -> Prometheus :9090 -> Grafana :3001
  cAdvisor + node-exporter -> Prometheus
  Uptime Kuma -> HTTP check on /api/health
```

Service	Address (dev)	Address (prod)	Role
Application	`http://localhost:8081`	`http://localhost`	User interface
API	`http://localhost:3000`	proxied via Nginx	REST endpoints
API health	`http://localhost:3000/api/health`	`/api/health`	Liveness check
Metrics	`http://localhost:3000/metrics`	—	Prometheus scrape target
Prometheus	`http://localhost:9090`	—	Metrics collection
Grafana	`http://localhost:3001`	—	Dashboards
Uptime Kuma	`http://localhost:3002`	—	Uptime monitoring

The system is a conventional three-layer client/API architecture. The frontend owns presentation state and interaction. The backend owns authentication, persistence, moderation rules, external data ingestion, review aggregation, and recommendation scoring.

3. Repository Structure

```
bean-beacon/
├── bean-beacon-frontend/        React/Vite web client
│   ├── src/pages/                route-level screens
│   ├── src/components/           reusable UI and feature components
│   ├── src/hooks/                React Query hooks and shared UI logic
│   ├── src/lib/api.ts            typed fetch wrapper, auth storage, API methods
│   ├── src/data/shops.ts         fallback sample cafés and filter vocabulary
│   └── nginx/default.conf        production static hosting and API proxy
├── bean-beacon-backend/          Express/TypeScript API
│   └── src/
│       ├── routes/               HTTP route declarations
│       ├── controllers/          request/business orchestration
│       ├── services/             external integrations, recommendation logic
│       ├── models/               Mongoose schemas
│       ├── middleware/           JWT, optional-auth, admin, error handling
│       ├── config/               environment and MongoDB setup
│       └── metrics.ts            Prometheus metrics
├── monitoring/                   Prometheus and Grafana provisioning
├── docs/                         project documentation (this file)
├── docker-compose.yml            development stack
├── docker-compose.prod.yml       production stack
├── deploy.py                     pyinfra local deployment script
├── demo.ps1                      Windows demonstration script
└── .gitlab-ci.yml                validate / test / security / build pipeline
```

Frontend routes:

Route	Screen	Responsibility
`/`	`Index`	café discovery, filtering, recommendations, list/map views
`/manage`	`Manage`	authentication, café submission, listing management
`/cafe/:id`	`CafeProfile`	detailed café information and reviews
`/dashboard`	`Dashboard`	admin moderation, review administration, activity
`*`	`NotFound`	fallback for unknown routes

4. DevOps Architecture Diagrams

The full pipeline — developer to production, including GitLab CI/CD, the container environment, and the observability layer — is shown below in two equivalent renderings.

Pipeline stages at a glance:

```
Developer → GitLab CI/CD → GitLab Container Registry → Production Environment
            |                                          |
            1. Lint & Validate                         Nginx Reverse Proxy
            2. Test                                       ├── Frontend (React + Vite)
            3. Security (dependency/secret/image scan)    └── Backend (Node.js + Express)
            4. Build (Docker images, tag, push)                 └── MongoDB
            5. Deploy (pull, restart, health check)       Docker Compose (shared volumes & network)
                                                           Monitoring: Prometheus, Grafana,
                                                                       Node Exporter, cAdvisor
```

External services feeding the production environment: OpenStreetMap/Overpass API (map data), an optional email service (notifications, password reset — not yet implemented), and SSL certificates for HTTPS.

Key DevOps benefits: automated delivery, security scanning and secret management, real-time observability, container-based scalability, and documented/versioned reproducibility.

(See the two attached architecture images — light and dark variants — for the full visual diagram with all pipeline stages, container boundaries, and monitoring components.)

Part II — Getting Started

5. Prerequisites

- **Docker Desktop** with Docker Compose — orchestrates all services
- **Git** — versioning and GitLab CI/CD integration
- **Node.js 16+** and npm — validation outside Docker (build, lint, tests)
- **Python + pyinfra** — optional local automated deployment via `deploy.py`

Verify your installation:

```
docker --version          # Docker version 24+
docker compose version    # Compose v2+
git --version
node --version             # v16+
npm --version
```

6. Running with Docker Compose

This is the recommended workflow for the repository.

From the repository root:

```
cd "$HOME\Desktop\coffee-quest\bean-beacon"
```

Create the external network used by the monitoring stack, if it doesn't already exist:

```
docker network inspect bean-beacon_app-network *> $null
if ($LASTEXITCODE -ne 0) {
    docker network create bean-beacon_app-network
}
```

Validate the configuration and start the stack:

```
docker compose config --quiet
docker compose up -d --build
```

Check container status:

```
docker compose ps
```

Open:

- Application — `http://localhost:8081`
- API health — `http://localhost:3000/api/health`
- Prometheus — `http://localhost:9090`
- Grafana — `http://localhost:3001`
- Uptime Kuma — `http://localhost:3002`

Test the API directly:

```
Invoke-RestMethod "http://localhost:3000/api/health"
```

Expected response:

```
{
  "success": true,
  "message": "Bean Beacon API is running"
}
```

Useful commands:

```
docker compose ps
docker compose logs --tail 100 -f backend
docker compose logs --tail 100 -f frontend
docker compose restart backend frontend
docker compose stop
```

> **Preserving data:** Always use `docker compose stop` to shut the stack down. Avoid `docker compose down -v` — it permanently

> deletes the MongoDB and Grafana volumes.

Restarting after code changes

In development, the frontend and backend mount source files and typically hot-reload. If needed:

```
docker compose restart frontend backend
```

Running the production build locally

```
cd "$HOME\Desktop\coffee-quest\bean-beacon"
docker compose -f docker-compose.prod.yml up -d --build
```

Open the production frontend at `http://localhost`. For production frontend-only or backend-only changes:

```
docker compose -f docker-compose.prod.yml up -d --build frontend
docker compose -f docker-compose.prod.yml up -d --build backend
```

Stop production services without losing data:

```
docker compose -f docker-compose.prod.yml stop
```

> Keep development and production workflows separate — don't use the dev stack's `up -d --build` as a way to "restart" production.

7. Manual Setup (without Docker)

Frontend

```
cd bean-beacon-frontend
npm install
cp .env.example .env
npm run dev
```

Available at `http://localhost:8081`.

Backend

```
cd bean-beacon-backend
npm install
cp .env.example .env
npm run dev
```

Available at `http://localhost:3000`. Watch for `Server running on port 3000`.

MongoDB options

Option A — MongoDB Atlas (cloud): create a free cluster at mongodb.com/cloud/atlas, create a database user, and copy the connection string into `MONGODB_URI`.

Option B — Local install: - macOS: `brew install mongodb-community && brew services start mongodb-community` - Ubuntu: `sudo apt-get install mongodb && sudo systemctl start mongodb` - Windows: download from mongodb.com/try/download/community

Environment files

Frontend `.env`:

```
VITE_API_URL=http://localhost:3000/api
```

Backend `.env`:

```
MONGODB_URI=mongodb://localhost:27017/bean-beacon
JWT_SECRET=your-secret-key
PORT=3000
NODE_ENV=development
FRONTEND_URL=http://localhost:8081
```

Generate a secret:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

The example/default secret is for local development only — production requires a unique, random `JWT_SECRET` (see §22).

8. Tests and Validation

Backend:

```
cd bean-beacon-backend
npm ci
npm run build
npm test -- run    # Vitest
```

Frontend:

```
cd ..\bean-beacon-frontend
npm ci
npm run lint
npm run typecheck
npm run build
cd ..
```

Security scan:

```
docker run --rm -v "${PWD}:/path" zricethezav/gitleaks:latest detect --source="/path" --verbose --redact
```

Part III — Application Logic

9. Frontend Architecture

9.1 Bootstrap and routing

`src/main.tsx` mounts `App` into the root element. `App.tsx` sets up:

1. React Strict Mode;
2. a single TanStack Query client;
3. tooltip and toast providers;
4. browser routing (see route table in §3).

9.2 Data access and session handling

`src/lib/api.ts` is a small typed `fetch` wrapper:

- base URL from `VITE_API_URL`, defaulting to `/api`;
- sends JSON for every request;
- attaches the stored JWT as `Authorization: Bearer <token>`;
- expects responses shaped as `{ success, data, error }`;
- failed responses become a typed `ApiError` carrying the HTTP status.

The JWT and a small user object are stored in `localStorage` under `bean-beacon-token` and `bean-beacon-user`. A local listener set handles login/logout notification without a React auth context.

> **Security note:** this is simple and workable for an MVP, but `localStorage` makes the token readable by any JavaScript running on the

> page — an XSS-sensitive design. A hardened production setup would use secure, HttpOnly cookies plus CSRF protection instead. This is tracked as a known risk (§25).

9.3 Café query and fallback behavior

`useCafes()` (TanStack Query, key `cafes`) calls `GET /api/cafes/all`.

- API café records are converted to the frontend `CoffeeShop` shape.
- GeoJSON `[longitude, latitude]` is converted to `{ lat, lng }`.
- A shared fallback image is used everywhere, since there is no photo storage yet.
- If the API returns an empty list, six records from `src/data/shops.ts` are shown as sample data, marked `isSample`.

Filtering then runs client-side: category matching is normalized for case/accents, search matches café names, and attribute filters use AND semantics (every selected attribute in a group must be present).

9.4 Maps

- `CafeMap.tsx` renders Leaflet markers (OpenStreetMap tiles) for cafés and the user's own location.
- Marker popups and `CoffeeShopCard` both offer a Google Maps directions link — Google Maps is used only for directions, not as the map data source or renderer.
- The backend supports a location-radius API, but the home page's location button currently only reads browser geolocation and shows a toast — it does not yet call that radius endpoint. (Tracked as a gap, §25.)

9.5 Key user flows

Discovery

```
Index mounts → useCafes() fetches /api/cafes/all → records mapped to CoffeeShop
  → local category/name/attribute filters run → cards or map markers render
  → user opens profile, directions, phone link, or reviews
```

Add a café

```
Manage → unauthenticated user sees sign-in/sign-up form
  → login/register stores JWT + user metadata
  → form submits POST /api/cafes
  → backend geocodes the address if coordinates weren't supplied
  → regular-user listing is created as "pending"
  → query cache invalidated, UI refreshes
```

Review a café

```
CafeProfile or card review dialog → GET /api/cafes/:cafeId/ratings
  → authenticated user submits POST /api/cafes/:cafeId/ratings
  → backend creates or upserts the user's single rating
  → backend recalculates the café's average and count
  → frontend invalidates reviews and café queries
```

Admin moderation

```
Dashboard → admin JWT required by backend
  → fetch all cafés and all reviews in parallel
  → approve/reject café, or delete café/review
  → invalidate dashboard and café queries
```

10. Backend Architecture

10.1 Startup sequence

`src/index.ts`:

1. loads environment variables;
2. creates the Express app;
3. enables CORS for the configured frontend origin;
4. enables JSON/URL-encoded body parsing;

5. attaches request metrics middleware;
6. connects to MongoDB, exiting on failure;
7. mounts route modules;
8. exposes `/api/health` and `/metrics`;
9. attaches error and 404 handlers;
10. starts listening on the configured port.

Every successful response is wrapped as `{ success: true, data }` — the contract the frontend wrapper expects.

10.2 Middleware and authorization

Three auth modes:

- **`authMiddleware`** — requires and verifies a JWT.
- **`optionalAuthMiddleware`** — verifies a JWT if present, otherwise allows anonymous access.
- **`adminMiddleware`** — requires an authenticated user whose token role is `admin`.

JWT claims: `userId`, `email`, `role`. Passwords are hashed with bcryptjs in the `User` model's pre-save hook. Default token expiry is 7 days. The error handler converts Mongoose validation and duplicate-key errors into JSON API errors; route handlers also do explicit validation/authorization.

10.3 Persistence model

User — email (unique, lowercased), bcrypt hash, name, role, timestamps. Password excluded from normal queries.

Cafe — holds both imported and user-submitted cafés. Location is a GeoJSON `Point` with a `2dsphere` index. Fields include identity, address, contact info, café attributes, rating aggregate, owner, and moderation status. `osmId` is unique and sparse — imported OSM records dedupe cleanly without blocking user-submitted records that lack an OSM ID.

Favorite — `userId` + `cafeId`, with a compound unique index (one favorite per user/café pair).

Rating — `userId`, `cafeId`, score, review text, author name, timestamps. Compound unique index (one rating per user/café pair). The café's `userRating` and `reviewCount` are denormalized aggregates recalculated after add/update/delete.

10.4 Café discovery and external data

Two acquisition paths:

1. **`GET /api/cafes/all`** reads the application database directly and applies visibility rules (see below). Sorts by `createdAt` only; hard-capped at 200 results with no `page`/`limit` params.
2. **`GET /api/cafes?latitude=&longitude=&radius=`** first runs a MongoDB geospatial query. If fewer than 10 cafés are found, it queries Overpass, imports new OSM cafés, then re-queries MongoDB.

Overpass responses are cached in memory for 1 hour; the cache is process-local (lost on restart, not shared across replicas). Imported cafés get coordinates, name, address fragments, type, Wi-Fi/power flags, website, phone, and tags.

User-submitted addresses are geocoded via Nominatim when coordinates are omitted (also cached 1 hour). The backend sends a configurable user-agent with a 10-second timeout. A geocoding failure to resolve an address returns `422`; a temporary geocoding service failure returns `502`.

10.5 Moderation and visibility logic

- Regular users see every `status: approved` café, plus their own submissions regardless of status.
- Admins see every status.
- A regular user cannot self-approve: `createCafe` ignores any submitted status and always assigns `pending`. Admins can create approved records directly.
- Imported OSM cafés use the schema's `approved` default, since they have no owner-submission workflow.

11. Recommendation Logic

Recommendations are **deterministic token matching**, not machine learning.

For each candidate café, three optional match ratios are computed:

Preference	Matched against	Weight
Drink	café name, category, menu, coffee types	3
Atmosphere	café name, category, atmosphere, amenities	3
Location	café name, address	2

Text is normalized (accents stripped, lowercased, tokenized) and compared as unique token sets.

```
score = 3 × drinkMatch + 3 × atmosphereMatch + 2 × locationMatch
```

Only cafés scoring above zero are kept. Results are sorted by score, then rating, then name, and capped at **3**. The response includes a short explanation of which preference dimensions matched.

The frontend sends its currently-loaded café list (1–200 shops) to `POST /api/cafes/recommend` along with `drink` / `atmosphere` / `location` preferences (at least one required). This is a local preference matcher — no AI provider or API key is required, and recommendation quality is limited to whatever list the browser already has loaded; it is not an independent database search. The backend validates the shape of the submitted shops but does not independently reload café records from the database.

Part IV — API Reference

Base URL: `http://localhost:3000/api`

12. Authentication

Register

```
POST /auth/register
Content-Type: application/json

{ "email": "user@example.com", "password": "securePassword123", "name": "John Doe" }
```

`201 Created` → `{ success, data: { token, user: { id, email, name } } }` Errors: `400` missing fields / invalid email / already registered · `500` server error

Login

```
POST /auth/login
Content-Type: application/json

{ "email": "user@example.com", "password": "securePassword123" }
```

`200 OK` → `{ success, data: { token, user } }` Errors: `400` missing credentials · `401` invalid email or password · `500` server error

Get current user

```
GET /auth/me
Authorization: Bearer <token>
```

`200 OK` → `{ success, data: { id, email, name } }` Errors: `401` no/invalid token · `404` user not found

13. Cafés

Get nearby cafés

```
GET /cafes?latitude=40.7128&longitude=-74.0060&radius=5
Authorization: Bearer <token>   (optional)
```

Query params: `latitude` *(required)*, `longitude` *(required)*, `radius` *(optional, default 5 km)*. `200 OK` → array of café objects. Errors: `400` missing lat/lng · `500` server error.

Get all cafés

```
GET /cafes/all
```

Approved cafés for public/anonymous callers; a user's own pending/rejected submissions are included for that user; admins see every status. No pagination currently (see §25).

Search cafés

```
GET /cafes/search?query=espresso&latitude=40.7128&longitude=-74.0060
Authorization: Bearer <token>   (optional)
```

`query` *(required)*; `latitude` / `longitude` *(optional, for distance sorting)*.

Recommend cafés

```
POST /cafes/recommend
Content-Type: application/json

{
  "drink": "pour-over",
  "atmosphere": "quiet place to work",
  "location": "Old Town",
  "shops": [ { "id": "cafe-id", "name": "Quiet Corner", "address": "Old Town",
               "coffeeTypes": ["Pour-over"], "atmosphere": ["Quiet","Work-friendly"],
               "amenities": ["WiFi"] } ]
}
```

Each preference is optional individually, but at least one is required. `shops` accepts 1–200 entries; the response contains up to three matching IDs with short explanations. See §11.

Create a café

```
POST /cafes
Authorization: Bearer <token>
Content-Type: application/json

{ "name": "Quiet Corner", "address": "123 Main Street, Old Town", "category": "Café",
  "coffeeTypes": ["Pour-over"], "atmosphere": ["Quiet","Work-friendly"], "amenities": ["WiFi"] }
```

If `lat` / `lng` are omitted, the backend geocodes the address via Nominatim. New listings from non-admins are always created `pending`. Errors: `422` address not found · `502` geocoding service failure.

Get café details

```
GET /cafes/:id
Authorization: Bearer <token>   (optional)
```

`200 OK` → full café object. Errors: `404` not found.

Edit / delete a café

```
PATCH /cafes/:id         (owner or admin)
DELETE /cafes/:id        (owner or admin)
```

Approve / reject a submission (admin)

```
PATCH /cafes/:id/status
```

14. Favorites

> **Status: backend complete, frontend not wired up.** The model, controller, and routes below all work; nothing in the frontend currently calls them. See §24.

```
GET    /favorites                 (auth required) — list current user's favorites
POST   /favorites/{cafeId}        (auth required) — add a favorite
DELETE /favorites/{cafeId}        (auth required) — remove a favorite
GET    /favorites/{cafeId}/check  (auth optional) — check favorite state
```

`POST` errors: `400` already a favorite · `401` not authenticated · `404` café not found. `DELETE` errors: `401` not authenticated · `404` favorite not found.

15. Ratings & Reviews

```
GET  /cafes/{cafeId}/ratings                  (auth optional)
POST /cafes/{cafeId}/ratings                  (auth required)
PUT  /cafes/{cafeId}/ratings/{ratingId}       (auth required, owner)
DEL  /cafes/{cafeId}/ratings/{ratingId}       (owner or admin)
```

`POST` body: `rating` *(required, 1–5)*, `review` *(optional, ≤1000 chars)*. One rating per user/café pair (subsequent posts upsert). `PUT` body fields are all optional.

Errors: `400` invalid rating · `401` not authenticated · `403` editing/deleting another user's rating · `404` café or rating not found.

16. Admin

```
GET   /admin/reviews          — all reviews (max 500) for moderation
GET   /admin/users            — list users
POST  /admin/users            — create a user
PATCH /admin/users/:id        — edit user / change role
PATCH /admin/cafes/:id        — admin edit of any café
PATCH /cafes/:id/status       — approve / reject a submission
```

Passwords are never returned by the API. **Admin role changes require signing in again**, since the JWT must be reissued to carry the new role.

17. Complete Route Table

Method	Route	Auth	Description
GET	`/health`	public	Liveness check
GET	`/metrics`	public	Prometheus metrics
POST	`/auth/register`	public	Create account
POST	`/auth/login`	public	Login
GET	`/auth/me`	JWT	Current profile
GET	`/cafes/all`	optional	All visible cafés
GET	`/cafes`	optional	Geospatial radius + OSM import
GET	`/cafes/search`	optional	Text search
GET	`/cafes/:id`	optional	Café detail
POST	`/cafes`	JWT	Create café
POST	`/cafes/recommend`	public	Recommendation
PATCH	`/cafes/:id`	JWT	Edit café (owner/admin)
PATCH	`/cafes/:id/status`	admin	Approve / reject
POST	`/cafes/:id/photo`	JWT	*Not yet implemented — see §24*
DELETE	`/cafes/:id`	JWT	Delete café (owner/admin)
GET	`/cafes/:id/ratings`	optional	Café reviews
POST	`/cafes/:id/ratings`	JWT	Create/upsert review
PUT	`/cafes/:id/ratings/:rid`	JWT	Edit own review
DELETE	`/cafes/:id/ratings/:rid`	owner/admin	Delete review
GET	`/favorites`	JWT	My favorites
POST	`/favorites/:cafeId`	JWT	Add favorite
DELETE	`/favorites/:cafeId`	JWT	Remove favorite
GET	`/favorites/:cafeId/check`	optional	Check favorite
GET	`/admin/reviews`	admin	All reviews
GET	`/admin/users`	admin	List users
POST	`/admin/users`	admin	Create user
PATCH	`/admin/users/:id`	admin	Edit user/role
PATCH	`/admin/cafes/:id`	admin	Admin café edit

18. Error Handling

All errors follow:

```
{ "success": false, "error": "Error message describing what went wrong" }
```

Code	Meaning	Action
400	Bad Request	Check request format and required fields
401	Unauthorized	Provide a valid JWT
403	Forbidden	No permission for this action
404	Not Found	Resource doesn't exist
422	Unprocessable	e.g. address could not be geocoded
500	Server Error	Retry later or check logs
502	Bad Gateway	Upstream geocoding service failure

No rate limiting is currently implemented. CORS is enabled for the origin set in `FRONTEND_URL`.

Part V — CI/CD and Deployment

19. Git and GitLab Workflow

`main` must always stay deployable. Every change goes through a short-lived branch and a Merge Request with human review before merging.

```
git checkout -b feature/short-description
git add .
git commit -m "feat: description of the change"
git push origin feature/short-description
# → open a Merge Request in GitLab → review → merge into main → pipeline runs
```

Commit message conventions: `Add:`, `Fix:`, `Refactor:`, `Docs:`, `Test:`, or Conventional-Commits style (`feat:`, `fix:`) — be consistent within the repo.

> `demo.ps1` does **not** create commits or Merge Requests automatically — these stay in the GitLab workflow to guarantee human review of every change.

20. GitLab CI/CD Pipeline

`.gitlab-ci.yml` runs four sequential stages on a Runner tagged `bean-beacon`:

Stage	Contents
1. validate	Backend TypeScript build · frontend lint · frontend typecheck · frontend build
2. test	Backend unit tests (Vitest)
3. security	npm audit (frontend + backend, report-only) · Gitleaks secret scan (**blocking**)
4. build	Build backend + frontend Docker images, tag with commit SHA

If jobs sit pending, check that a Runner registered with exactly the `bean-beacon` tag is available. **Harbor is intentionally out of scope** — images are built but not pushed to a private registry. Deployment uses the local production Compose file instead.

21. Production Deployment

Local production stack

```
$env:JWT_SECRET = "replace-with-a-long-random-secret"
docker compose -f docker-compose.prod.yml up -d --build
```

Accessible at `http://localhost`. Equivalent via pyinfra:

```
pyinfra @local deploy.py
```

Hosted options (if deploying beyond local)

Frontend — Vercel (recommended for Vite), Netlify, or self-hosted via Nginx with `try_files $uri /index.html` and `/api/` proxied to the backend.

Backend — Render, Railway, Heroku, or a self-hosted VPS with PM2 (`pm2 start dist/index.js --name bean-beacon-api`) behind an Nginx reverse proxy.

Database — MongoDB Atlas (recommended) or self-hosted MongoDB.

Each hosted backend needs the same environment variables as below, plus HTTPS (Let's Encrypt via `certbot --nginx` for self-hosted options).

Post-deployment checklist

- [ ] Frontend accessible at the target domain
- [ ] `GET /api/health` responds
- [ ] Registration and login work
- [ ] Geolocation permission works; cafés load on the map
- [ ] Search, filters, favorites, and reviews work

- ☐ HTTPS enabled on frontend and backend
- ☐ Environment variables set correctly, including a unique `JWT_SECRET`
- ☐ MongoDB backups scheduled
- ☐ `FRONTEND_URL` matches the real production origin (CORS)

22. Environment Variables

> **`JWT_SECRET` is mandatory in production.** Generate a unique value per environment — never reuse the Docker development default. The production Compose file enforces this; non-Compose deployments must enforce it themselves (tracked as a risk, §25).

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Frontend (`.env.production`):

```
VITE_API_URL=https://api.yourdomain.com/api
```

Backend (`.env.production`):

```
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/bean-beacon
JWT_SECRET=generate-long-random-secure-string-here
JWT_EXPIRE=7d
PORT=3000
NODE_ENV=production
FRONTEND_URL=https://yourdomain.com
OVERPASS_API_URL=https://overpass-api.de/api/interpreter
NOMINATIM_API_URL=https://nominatim.openstreetmap.org
```

Café photo uploads (once implemented, see §24) are stored under the backend's `uploads` directory — deployments with ephemeral filesystems must mount persistent storage there. The production Compose file provides a named volume for this.

Part VI — Observability

23. Monitoring Stack

Tool	Address	Role
Prometheus	`localhost:9090`	Scrapes `/metrics` every 15s, plus cAdvisor and node-exporter
Grafana	`localhost:3001`	Dashboards; Prometheus pre-provisioned as data source; data in `grafana_data` volume
Uptime Kuma	`localhost:3002`	HTTP monitor on `/api/health`, 60s interval, 2 retries

Metrics exposed by the backend (via `prom-client`, prefixed `bean_beacon_`):

- `bean_beacon_http_request_duration_seconds` (histogram)
- `bean_beacon_http_requests_total` (counter)
- both labeled by HTTP method, resolved route, and status code
- default Node.js process metrics are also exposed under the same prefix

Unmatched paths are recorded under a fixed `unmatched` route label to avoid unbounded metric cardinality.

Uptime Kuma setup: create an HTTP(s) monitor using the Docker service name (not `localhost`, since Uptime Kuma runs in the same Compose network):

```
Name:     Bean Beacon API
URL:      http://backend:3000/api/health
Interval: 60 seconds
Retries:  2
```

To test: `docker compose stop backend` then `docker compose start backend` — the monitor should go red, then recover.

Grafana password reset (without losing the volume):

```
docker compose exec grafana grafana cli admin reset-admin-password "NewStrongPassword!"
```

> Prometheus and Grafana are included in the **development** Compose file but not currently in `docker-compose.prod.yml` — see §25.

Part VII — Reference

24. Feature Improvement Backlog

Grounded in a direct read of the current backend and frontend code, not assumptions. Split into near-complete features that just need finishing, and features that don't exist yet.

Quick wins — backend already built

Favorites — the `Favorite` model, `favoriteController.ts` (get/add/remove), and the `GET` / `POST` / `DELETE /api/favorites` routes all exist and work. Nothing in the frontend calls them: no `favoritesApi` client methods, no save button wired up, no favorites list page. *To finish:* add `favoritesApi` methods to `api.ts`, a toggleable heart/save icon on café cards, and a "My Favorites" view (new page or filter tab).

Photo uploads — the `Cafe` model already has `photoPath` and `galleryPaths` fields, but there is no upload endpoint, no multer (or equivalent) middleware, and no file input in `Manage.tsx`'s submission form. A café photo can currently only exist if someone edits the database directly. *To finish:* add an upload endpoint (local disk storage to start; S3/Cloudinary later if needed), wire a file input into the submission form, and render the photo in `CafeProfile.tsx` and on café cards.

Missing entirely

Forgot password / reset flow — `authRoutes.ts` only has `register`, `login`, and `me`. There is currently no way back into an account except re-registering with a different email. This also connects to the security grading criteria, since a reset flow is a natural place to demonstrate secure, short-lived, single-use token handling (never emailing the password itself). *Scope:* `POST /api/auth/forgot-password` (issues a token, emails a link — needs an email service, even a free tier like Resend, or a console-log fallback in dev) and `POST /api/auth/reset-password` (verifies the token, sets the new password).

Owner notification on café approval/rejection — after submitting a café, a user's only way to find out whether it was approved is to keep checking `Manage.tsx` themselves. No email, no in-app notification. *Scope:* simplest version is in-app only — a notification bell/badge reading unread status changes on the user's own submissions (no new infrastructure required). An email version is a larger iteration of the same idea and can share infrastructure with the password-reset email service above.

Distance sorting + pagination on the main café list — `GET /api/cafes/all`, which actually powers the merged app's main list, sorts by `createdAt` only and hard-caps at 200 results with no `page` / `limit` params. The `$near` geospatial distance sort that already exists on the older `getCafesByLocation` endpoint isn't used here. *Scope:* add `page` / `limit` query params to `listAllCafes`, and when `lat` / `lng` are supplied, sort by distance using the `2dsphere` index already present on the `Cafe` model.

Suggested order: Favorites and photo uploads are the cheapest wins — the hard backend work for favorites is already done, and photo uploads is mostly plumbing. Forgot-password and owner notifications are both naturally bigger (they need an email service), so building them together once that infrastructure exists is efficient. Distance sorting + pagination is self-contained backend work with no new dependencies and can slot in anywhere.

25. Known Risks and Gaps

Observations drawn from the current source code, not assumptions.

#	Risk / gap	Criticality
1	Frontend doesn't call the favorites API, despite backend support existing	Medium
2	Home page location button gets browser coordinates but never calls `/api/cafes` (radius endpoint)	Low
3	JWT stored in `localStorage` — readable by any JS on the page (XSS-sensitive)	**High**
4	No rate limiting; no pagination on most public views	Medium
5	Rating aggregate update is a read-then-write, not transactional — possible race condition under concurrent reviews	Low
6	`JWT_SECRET` has a development fallback in app config; non-Compose deployments must self-enforce a real secret	**High**
7	Prometheus/Grafana are in dev Compose but not in `docker-compose.prod.yml`	Low
8	Overpass/Nominatim cache is in-process memory — lost on restart, not shared across replicas	Low
9	No centralized schema validation library (e.g. Zod/Joi) on the backend beyond hand-written checks and Mongoose validation	Medium
10	No forgot-password flow	High (UX)
11	Recommendation endpoint trusts café data supplied by the browser; the backend validates shape but doesn't independently reload records from the database	Low–Medium
12	No café photo storage despite model fields reserved for it; frontend always shows a fallback image	Medium

If any other documentation conflicts with the running system, treat the source code and Compose files as authoritative — always re-verify directory names, ports, and service behavior against them directly.

26. Troubleshooting

No containers appear with `docker compose ps` The stack isn't started: `docker compose up -d --build`

API returns "Route not found" at `http://localhost:3000/` Expected — the root route isn't defined. Use `http://localhost:3000/api/health` instead.

Grafana rejects the password Reset without deleting the volume:

```
docker compose exec grafana grafana cli admin reset-admin-password "NewStrongPassword!"
```

MongoDB or Grafana data disappeared Always use `docker compose stop`. Avoid `docker compose down -v` — it permanently deletes persistent volumes.

GitLab jobs stay pending Check that a Runner is registered with exactly the tag `bean-beacon`.

Cafés aren't loading Verify Overpass API reachability, browser geolocation permission, and that the backend can reach `overpass-api.de`. Check `docker compose logs -f backend`.

Frontend deployed but backend times out Check `VITE_API_URL`, confirm the backend is running and reachable, and check backend CORS configuration.

Can't log in after deployment Check `JWT_SECRET` is set, confirm the database is reachable, and check the browser console for errors.

27. Final Deliverables

Deliverable	Location / verification
Frontend application	`bean-beacon-frontend/`
Backend API	`bean-beacon-backend/`
Database	`mongo` service in Compose
Dev Compose file	`docker-compose.yml`
Prod Compose file	`docker-compose.prod.yml`
GitLab pipeline	`.gitlab-ci.yml`
Health check	`GET /api/health`
Backend tests	`bean-beacon-backend/tests/`
Local deployment automation	`deploy.py` (pyinfra)
Windows demo script	`demo.ps1`
Monitoring	Prometheus · Grafana · Uptime Kuma
This documentation	`docs/FINAL_DOCUMENTATION.md`

This document consolidates all prior Bean Beacon documentation (architecture notes, API reference, setup/run/deployment guides, both cahier des charges versions, and the feature improvement backlog) into a single current reference, verified against the implementation where sources disagreed.